Кооперация процессов и логическая организация взаимодействия

Тема 6

Кооперация процессов и основные аспекты ее логической организации

Содержание:

- Взаимодействие процессов
- Способы обмена информацией
- Каналы связи
- Нити исполнения (threads)

Что такое взаимодействие процессов

Взаимодействующие процессы — это процессы, которые влияют на поведение друг друга через обмен информацией.

Они могут выполняться:

- псевдопараллельно на одном компьютере
- параллельно на разных компьютерах

Операционная система должна:

- изолировать процессы
- обеспечивать безопасное взаимодействие

Независимые и кооперативные процессы

Независимые процессы

- не влияют друг на друга
- не обмениваются информацией

Кооперативные процессы

- обмениваются данными
- изменяют поведение в зависимости от полученной информации

Причины кооперации процессов

Процессы взаимодействуют по нескольким причинам:

1. Повышение скорости работы

- параллельное выполнение задач
- использование нескольких процессоров

2. Совместное использование данных

- общая база данных
- общий файл

3. Модульная архитектура

- система состоит из отдельных модулей

4. Удобство пользователя

- одновременная работа редактора и отладчика

Категории средств обмена информацией

Существует **3 основных типа средств связи** между процессами.

1. Сигнальные средства

передают минимальную информацию

один бит: **да / нет**

Используются для уведомлений о событиях.

2. Канальные средства

Передача информации через **каналы связи**

Примеры:

- pipe
- FIFO
- sockets
- очереди сообщений

Объем передаваемых данных ограничен пропускной способностью канала.

Категории средств обмена информацией

3. Разделяемая память

Процессы используют общую область памяти.

Преимущества:

- высокая скорость обмена
- большой объем данных

Недостаток:

- требуется синхронизация процессов

Это самый быстрый способ взаимодействия на одном компьютере.

Адресация при обмене данными

При передаче данных необходимо указать адрес получателя.
Существует два вида адресации:

Прямая адресация

процессы обращаются непосредственно друг к другу

Варианты:

- симметричная
- Асимметричная

Непрямая адресация

данные передаются через **промежуточный объект**

Например:

- очередь сообщений
- буфер
- доска объявлений

Преимущества:

- гибкость
- несколько процессов могут использовать один канал

Буферизация каналов

Канал связи может иметь буфер.
Типы буферов:
1. Нулевой буфер
- отправитель ждёт получателя
2. Ограниченный буфер
- хранит n сообщений
3. Неограниченный буфер (теоретически)
- отправитель никогда не ждёт

Модели передачи данных

Существует две модели передачи данных.

1. Поток данных (stream)

данные передаются как **последовательность байтов**

Примеры:

- pipe
- FIFO

Получатель не знает структуру данных.

2. Модель сообщений

данные разбиваются на **сообщения**

Сообщение может содержать:

- данные
- отправителя
- получателя

Примеры:

- очереди сообщений
- sockets

Надежность связи

Связь считается надежной, если:

1. нет потери информации
2. нет повреждения данных
3. не появляется лишняя информация
4. сохраняется порядок сообщений

Методы повышения надежности:

- контрольные суммы
- подтверждение получения
- повторная отправка
- нумерация сообщений

Нити исполнения (Threads)

Нить исполнения (thread) — это часть процесса.

Нити одного процесса:

- используют общий код
- используют общую память
- используют общие ресурсы

Но каждая нить имеет:

- свой стек
- свой счетчик команд
- свои регистры

Нити исполнения (Threads)

Ввести массив a

Ввести массив b

Ввести массив c

a = a + b

c = a + c

Вывести массив c

Пример 1.

Ввести массив a	
Ожидание окончания операции ввода	
Ввести массив b	
Ожидание окончания операции ввода	
Ввести массив c	
Ожидание окончания операции ввода	a = a + b
c = a + c	
Вывести массив c	

Пример 2.

Нити исполнения (Threads)

Процесс 1	Процесс 2
Ввести массив a	Ожидание ввода массивов a и b
Ожидание окончания операции ввода	
Ввести массив b	
Ожидание окончания операции ввода	
Ввести массив c	
Ожидание окончания операции ввода	a = a + b
c = a + c	
Вывести массив c	
Ожидание окончания операции вывода	

Пример 3.

Процесс 1	Процесс 2
Создать процесс 2	
Переключение контекста	
	Выделение общей памяти
	Ожидание ввода a и b
Переключение контекста	
Выделение общей памяти	
Ввести массив a	
Ожидание окончания операции ввода	
Ввести массив b	
Ожидание окончания операции ввода	
Ввести массив c	
Ожидание окончания операции ввода	
Переключение контекста	
	a = a + b
Переключение контекста	
c = a + c	
Вывести массив c	
Ожидание окончания операции вывода	

Пример 4.

Преимущества нитей

Нити позволяют:

- выполнять задачи параллельно
- ускорять выполнение программ
- уменьшать время ожидания

Преимущества:

- быстрое создание
- быстрое переключение контекста
- эффективное использование процессора

Заключение

Кооперация процессов необходима для:

- ускорения вычислений
- совместного использования данных
- удобства работы

Основные способы взаимодействия:

- сигналы
- каналы связи
- разделяемая память

Для повышения эффективности используются **нити исполнения**, позволяющие параллельно выполнять части программы.